MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSS 2212)

TITLE

GigLess

SUBMITTED
BY

Sn. No.	Name	Reg. No.	Roll. No.
1	Vedhas B Achar	240968008	04
2	Danish Garg	240968296	29
3	Vaishnavi Bhandary	240968025	06

School of Computer Engineering
Manipal Institute of Technology, Manipal.
April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Manipal
04/04/2026

CERTIFICATE

This is to certify that the project titled **GigLess Title** is a record of the bonafide work done by **Student(s) (Reg. No. 240968008, 240968296, 240968025)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in ----------ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Roopalakshmi R, Associate Professor, SCE.**

2. **Prof. Giridhar N S , Assistant Professor-Selection Grade, SCE.**

TABLE OF CONTENTS

ABSTRACT

CHAPTER 1: INTRODUCTION

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

CHAPTER 3: METHODOLOGY

CHAPTER 4: ER Diagram & Relational Tables along with sample data

CHAPTER 5: DDL commands & PL/SQL Procedures / Functions / Triggers

CHAP TER 6: RESULTS & SNAPSHOTS

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

CHAPTER 8: REFERENCES

ABSTRACT

GigLess is a full-stack campus freelancing marketplace designed to connect student freelancers with customers within a university ecosystem. The platform enables students to offer and discover services such as design, development, writing, and tutoring, while providing a structured workflow for ordering, communication, and reviews. Built using Django REST Framework on the backend and React (Vite) on the frontend, GigLess supports JWT-based authentication, role-separated accounts, real-time WebSocket chat, and a complete order lifecycle. The system uses a relational database (SQLite for development, MySQL for production) with a well-normalized schema covering users, services, orders, reviews, conversations, and messages. This report details the motivation, objectives, architecture, database design, DDL implementation, and conclusions drawn from building GigLess.

CHAPTER 1: INTRODUCTION

The gig economy has transformed how people exchange skills and services. However, within university campuses, there exists no dedicated, trusted platform where students can offer freelance services to peers in a structured and secure manner. Students with skills in programming, graphic design, content writing, or academic tutoring often have no efficient channel to monetize these abilities or find help from fellow students.

GigLess addresses this gap by providing a campus-focused freelancing marketplace. The name "GigLess" reflects the platform's intent to reduce the friction and barriers typically associated with gig work — making it easier and more accessible for students. The application allows users to register either as a **Freelancer** or a **Customer**, browse and post services, place orders, communicate via real-time chat, and leave reviews once a service is delivered.

The platform is built as a modern web application with a clear separation between frontend and backend. The backend is developed using **Django 5** and **Django REST Framework**, with **SimpleJWT** for token-based authentication and **Django Channels** for WebSocket support. The frontend is built with **React 19** and **Vite**, consuming the backend REST API. Data is persisted in a relational database — SQLite during development and MySQL in production environments.

This report documents the end-to-end design and implementation of GigLess, covering problem identification, system objectives, methodology, database schema, SQL implementation, and conclusions.

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Students on university campuses possess a wide variety of skills but lack a dedicated, trustworthy platform to offer or procure services from fellow students. Existing general-purpose freelancing platforms (such as Fiverr or Upwork) are not tailored to campus contexts — they lack community trust, are cost-prohibitive for small student transactions, and offer no campus-specific filtering or peer verification. This results in:

- Skilled students being unable to monetize their abilities within their community.
- Students needing small, affordable services having no reliable peer-to-peer option.
- No structured mechanism for order tracking, communication, or dispute resolution among students.

2.2 Objectives

The primary objectives of GigLess are:

1. Role-based user management — Support two distinct user roles (Freelancer and Customer) with role-specific permissions and capabilities.
2. Service marketplace — Allow freelancers to create, categorize, price, and manage service listings that customers can search, filter, and sort.
3. Order lifecycle management — Implement a structured order flow from placement (pending) through active work (in_progress) to delivery (completed), with role-based status transition rules.

4. Review and rating system — Enable customers to review completed orders, with automatic aggregation of freelancer ratings to ensure marketplace trust.
5. Real-time communication — Provide a REST + WebSocket hybrid chat system scoped to conversation participants, enabling direct messaging within the context of an order.
6. Secure authentication — Implement JWT-based authentication with HTTP-only cookies to protect user sessions and sensitive endpoints.
7. Scalable database design — Design a normalized relational schema that supports all above features and can be deployed on both SQLite and MySQL.

CHAPTER 3: METHODOLOGY

3.1 Development Approach

GigLess follows a feature-driven, iterative development approach. The system was broken down into discrete functional modules — authentication, services, orders, reviews, and chat — each developed and tested independently before integration.

3.2 System Architecture

The application follows a decoupled client-server architecture:

- Backend: Django REST Framework serves a JSON API consumed by the frontend. Django Channels adds WebSocket support for real-time chat, served via Daphne (an ASGI server).
- Frontend: A React single-page application communicates with the backend via Axios over HTTP and the browser's native WebSocket API.
- Database: A relational database (SQLite in development, MySQL in production) stores all persistent data. Django's ORM is used for all database interactions, with schema managed through migrations.

3.3 Authentication & Security

Authentication is handled using SimpleJWT with two tokens:

- access_token — short-lived, stored in HTTP-only cookies, sent with each API request.
- refresh_token — longer-lived, used to silently obtain new access tokens.

HTTP-only cookies prevent JavaScript-based token theft. Role-based access control (RBAC) is enforced at the view level using Django REST Framework's permission classes.

3.4 Data Modelling

The data model consists of five core Django apps, each with its own models:

- accounts — Custom User model extending Django's AbstractUser.
- marketplace_services — Category and Service models.
- orders — Order and Review models with lifecycle logic.
- chat — Conversation and Message models for real-time messaging.

Each model maps directly to a relational database table. Foreign key constraints enforce referential integrity. Business logic (e.g., rating aggregation, conversation participant ordering) is implemented in model save() methods.

3.5 API Design

The REST API is organized under the /api base prefix with sub-routers for each module:

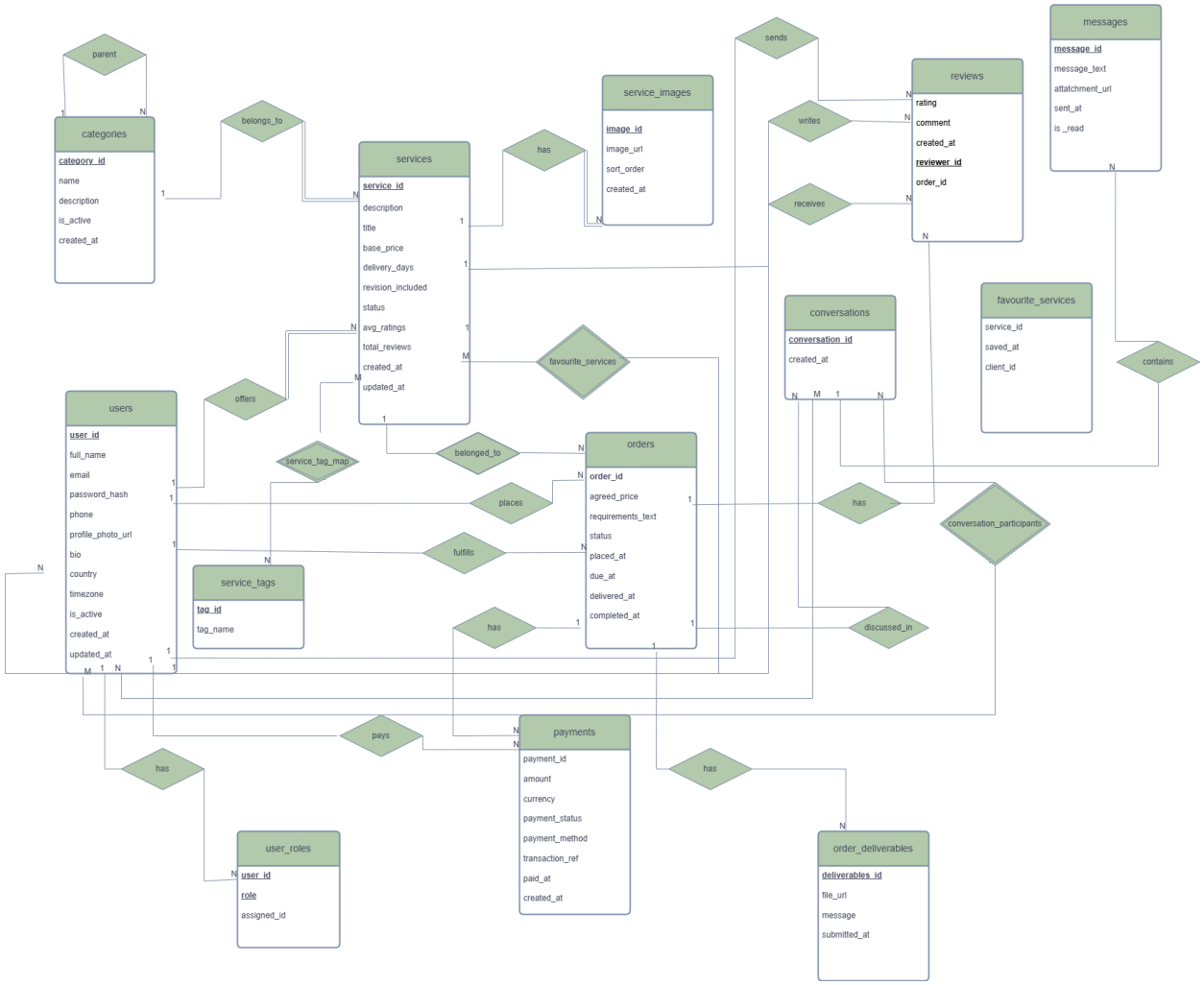
- /api/auth/ — Registration, login, logout, token refresh, profile management.
- /api/services/ — CRUD for services with filtering by keyword, category, price range, and rating.
- /api/orders/ — Order creation, status transitions (role-enforced), and review submission.
- /api/chat/ — Conversation creation, message history, WebSocket token issuance.

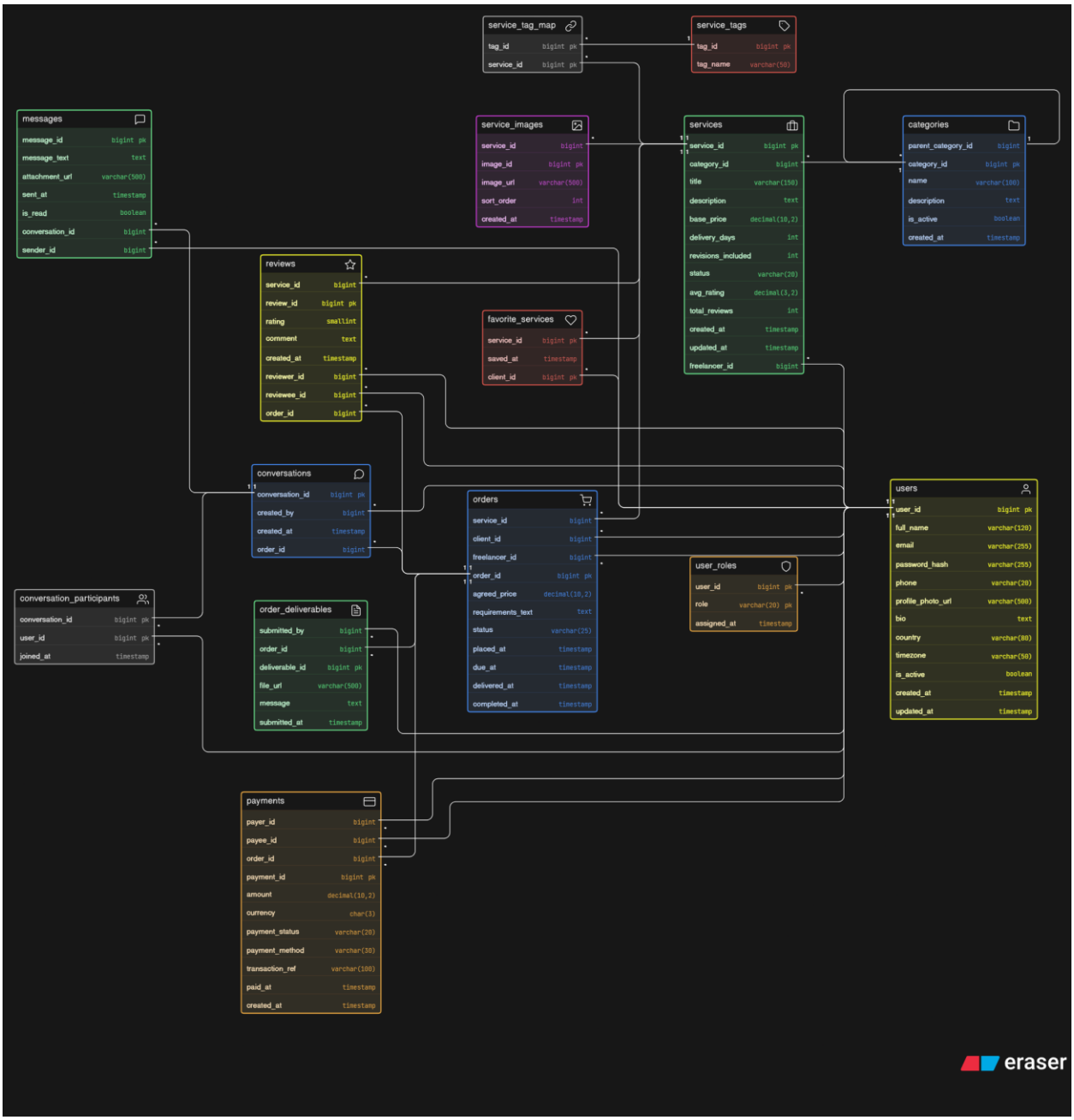
WebSocket connections are established at /ws/chat/{conversation_id}/?token=<jwt>, with the JWT validated on connection.

3.6 Tech Stack Summary

Layer	Technology
Backend Framework	Django 5, Django REST Framework
Authentication	SimpleJWT (HTTP-only cookies)
Real-time	Django Channels, Daphne (ASGI)
Frontend	React 19, Vite 8, React Router, Axios
Database (dev)	SQLite
Database (prod)	MySQL

CHAPTER 4: ER Diagram & Relational Tables along with sample data




service_tag_map
tag_id bigint pk
service_id bigint pk
service_tags
tag_id bigint pk
tag_name varchar(50)
messages
message_id bigint pk
message_text text
attachment_url varchar(500)
sent_at timestamp
is_read boolean
conversation_id bigint
sender_id bigint
service_images
service_id bigint
image_id bigint pk
image_url varchar(500)
sort_order int
created_at timestamp
services
service_id bigint pk
category_id bigint
title varchar(150)
description text
base_price decimal(10,2)
delivery_days int
revisions_included int
status varchar(20)
avg_rating decimal(3,2)
total_reviews int
created_at timestamp
updated_at timestamp
freelancer_id bigint
categories
parent_category_id bigint
category_id bigint pk
name varchar(100)
description text
is_active boolean
created_at timestamp
reviews
service_id bigint
review_id bigint pk
rating smallint
comment text
created_at timestamp
reviewer_id bigint
reviewee_id bigint
order_id bigint
favorite_services
service_id bigint pk
saved_at timestamp
client_id bigint pk
conversations
conversation_id bigint pk
created_by bigint
created_at timestamp
order_id bigint
orders
service_id bigint
client_id bigint
freelancer_id bigint
order_id bigint pk
agreed_price decimal(10,2)
requirements_text text
status varchar(25)
placed_at timestamp
due_at timestamp
delivered_at timestamp
completed_at timestamp
users
user_id bigint pk
full_name varchar(120)
email varchar(255)
password_hash varchar(255)
phone varchar(20)
profile_photo_url varchar(500)
bio text
country varchar(80)
timezone varchar(50)
is_active boolean
created_at timestamp
updated_at timestamp
user_roles
user_id bigint pk
role varchar(20) pk
assigned_at timestamp
conversation_participants
conversation_id bigint pk
user_id bigint pk
joined_at timestamp
order_deliverables
submitted_by bigint
order_id bigint
deliverable_id bigint pk
file_url varchar(500)
message text
submitted_at timestamp
payments
payer_id bigint
payee_id bigint
order_id bigint
payment_id bigint pk
amount decimal(10,2)
currency char(3)
payment_status varchar(20)
payment_method varchar(30)
transaction_ref varchar(100)
paid_at timestamp
created_at timestamp
eraser

CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

5.1 Overview

GigLess uses MySQL as its production database. The schema is auto-generated by Django's ORM via migrations, but the equivalent DDL is documented below. Supplementary stored routines (procedures and functions) and triggers enforce business rules at the database level.

Assumptions: MySQL 8.0+, InnoDB engine, utf8mb4 charset, all primary keys are BIGINT AUTO_INCREMENT.

5.2 Database Creation

```
CREATE DATABASE IF NOT EXISTS gigless
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE gigless;
```

5.3 DDL — Table Definitions

5.3.1 accounts_user (Custom User Table)

```
CREATE TABLE IF NOT EXISTS accounts_user (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  password      VARCHAR(128)   NOT NULL,
  last_login    DATETIME(6)    NULL,
  is_superuser  TINYINT(1)     NOT NULL DEFAULT 0,
  username      VARCHAR(150)   NOT NULL,
  first_name    VARCHAR(150)   NOT NULL DEFAULT '',
  last_name     VARCHAR(150)   NOT NULL DEFAULT '',
  email         VARCHAR(254)   NOT NULL,
  is_staff      TINYINT(1)     NOT NULL DEFAULT 0,
  is_active     TINYINT(1)     NOT NULL DEFAULT 1,
```

```
  date_joined   DATETIME(6)    NOT NULL,
  name          VARCHAR(120)   NOT NULL,
  role          VARCHAR(20)    NOT NULL,
  bio           LONGTEXT       NOT NULL DEFAULT '',
  skills        LONGTEXT       NOT NULL DEFAULT '',
  profile_photo VARCHAR(100)   NULL DEFAULT NULL,
  rating        DECIMAL(3,2)   NOT NULL DEFAULT 0.00,
  rating_count  INT UNSIGNED   NOT NULL DEFAULT 0,
  CONSTRAINT uk_accounts_user_username UNIQUE (username),
  CONSTRAINT uk_accounts_user_email    UNIQUE (email)
) ENGINE=InnoDB;
```

5.3.2 accounts_user_groups (M2M — User to Group)

```
CREATE TABLE IF NOT EXISTS accounts_user_groups (
  id       BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id  BIGINT NOT NULL,
  group_id INT    NOT NULL,
  CONSTRAINT fk_aug_user  FOREIGN KEY (user_id)  REFERENCES accounts_user(id) ON
DELETE CASCADE,
  CONSTRAINT  fk_aug_group  FOREIGN  KEY  (group_id)  REFERENCES  auth_group(id)
ON DELETE CASCADE,
  CONSTRAINT uk_aug_user_group UNIQUE (user_id, group_id)
) ENGINE=InnoDB;
```

5.3.3 accounts_user_user_permissions (M2M — User to Permission)

```
CREATE TABLE IF NOT EXISTS accounts_user_user_permissions (
  id           BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id      BIGINT NOT NULL,
  permission_id INT    NOT NULL,
  CONSTRAINT  fk_auup_user  FOREIGN  KEY  (user_id)  REFERENCES
accounts_user(id)  ON DELETE CASCADE,
  CONSTRAINT  fk_auup_permission  FOREIGN  KEY  (permission_id)  REFERENCES
auth_permission(id) ON DELETE CASCADE,
  CONSTRAINT uk_auup_user_perm  UNIQUE (user_id, permission_id)
) ENGINE=InnoDB;
```

5.3.4 marketplace_services_category

```
CREATE TABLE IF NOT EXISTS marketplace_services_category (
  id   BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  CONSTRAINT uk_msc_name UNIQUE (name)
) ENGINE=InnoDB;
```

5.3.5 marketplace_services_service

```
CREATE TABLE IF NOT EXISTS marketplace_services_service (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  freelancer_id BIGINT        NOT NULL,
  title         VARCHAR(180)  NOT NULL,
  description   LONGTEXT      NOT NULL,
  image         VARCHAR(100)  NULL DEFAULT NULL,
  category_id   BIGINT        NOT NULL,
  price         DECIMAL(10,2) NOT NULL,
  delivery_time INT UNSIGNED  NOT NULL,
  created_at    DATETIME(6)   NOT NULL,
  updated_at    DATETIME(6)   NOT NULL,
  CONSTRAINT fk_mss_freelancer FOREIGN KEY (freelancer_id)
    REFERENCES accounts_user(id) ON DELETE CASCADE,
  CONSTRAINT fk_mss_category FOREIGN KEY (category_id)
    REFERENCES marketplace_services_category(id) ON DELETE RESTRICT
) ENGINE=InnoDB;
```

5.3.6 orders_order

```
CREATE TABLE IF NOT EXISTS orders_order (
  id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
  customer_id         BIGINT      NOT NULL,
  service_id          BIGINT      NOT NULL,
  order_date          DATETIME(6) NOT NULL,
  status              VARCHAR(20) NOT NULL DEFAULT 'pending',
  dummy_payment_status VARCHAR(20) NOT NULL DEFAULT 'paid',
  CONSTRAINT fk_oo_customer FOREIGN KEY (customer_id)
    REFERENCES accounts_user(id) ON DELETE CASCADE,
  CONSTRAINT fk_oo_service FOREIGN KEY (service_id)
    REFERENCES marketplace_services_service(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

5.3.7 orders_review

```
CREATE TABLE IF NOT EXISTS orders_review (
  id         BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id   BIGINT          NOT NULL,
  rating     SMALLINT UNSIGNED NOT NULL,
  comment    LONGTEXT        NOT NULL DEFAULT '',
  created_at DATETIME(6)     NOT NULL,
  CONSTRAINT uk_or_order UNIQUE (order_id),
  CONSTRAINT fk_or_order FOREIGN KEY (order_id)
```

```
  REFERENCES orders_order(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

5.3.8 chat_conversation

```
CREATE TABLE IF NOT EXISTS chat_conversation (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_one_id BIGINT      NOT NULL,
  user_two_id BIGINT      NOT NULL,
  order_id    BIGINT      NULL DEFAULT NULL,
  created_at  DATETIME(6) NOT NULL,
  updated_at  DATETIME(6) NOT NULL,
  CONSTRAINT fk_cc_user_one FOREIGN KEY (user_one_id)
    REFERENCES accounts_user(id) ON DELETE CASCADE,
  CONSTRAINT fk_cc_user_two FOREIGN KEY (user_two_id)
    REFERENCES accounts_user(id) ON DELETE CASCADE,
  CONSTRAINT fk_cc_order FOREIGN KEY (order_id)
    REFERENCES orders_order(id) ON DELETE SET NULL,
  CONSTRAINT uniq_conversation_pair_order UNIQUE (user_one_id, user_two_id, order_id)
) ENGINE=InnoDB;
```

5.3.9 chat_message

```
CREATE TABLE IF NOT EXISTS chat_message (
  id              BIGINT AUTO_INCREMENT PRIMARY KEY,
  conversation_id BIGINT      NOT NULL,
  sender_id       BIGINT      NOT NULL,
  content         LONGTEXT    NOT NULL,
  is_read         TINYINT(1)  NOT NULL DEFAULT 0,
  created_at      DATETIME(6) NOT NULL,
  CONSTRAINT fk_cm_conversation FOREIGN KEY (conversation_id)
    REFERENCES chat_conversation(id) ON DELETE CASCADE,
  CONSTRAINT fk_cm_sender FOREIGN KEY (sender_id)
    REFERENCES accounts_user(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

5.4 Indexes

```
CREATE INDEX ix_mss_freelancer ON marketplace_services_service (freelancer_id);
CREATE INDEX ix_mss_category   ON marketplace_services_service (category_id);
CREATE INDEX ix_mss_price      ON marketplace_services_service (price);

CREATE INDEX ix_oo_customer ON orders_order (customer_id);
CREATE INDEX ix_oo_service  ON orders_order (service_id);
CREATE INDEX ix_oo_status   ON orders_order (status);

CREATE INDEX ix_or_order ON orders_review (order_id);
```

```
CREATE INDEX ix_cc_order ON chat_conversation (order_id);
CREATE INDEX ix_cm_conversation_created ON chat_message (conversation_id, created_at);
```

5.5 Stored Function

5.5.1 fn_freelancer_avg_rating — Average rating for a freelancer

```
DELIMITER $$

CREATE FUNCTION fn_freelancer_avg_rating(p_freelancer_id BIGINT)
RETURNS DECIMAL(3,2)
READS SQL DATA
BEGIN
  DECLARE v_avg DECIMAL(3,2);

  SELECT IFNULL(ROUND(AVG(r.rating), 2), 0.00)
    INTO v_avg
    FROM orders_review r
    JOIN orders_order  o ON o.id = r.order_id
    JOIN marketplace_services_service s ON s.id = o.service_id
   WHERE s.freelancer_id = p_freelancer_id;

  RETURN v_avg;
END$$

DELIMITER ;
```

5.6 Stored Procedures

5.6.1 pr_create_order — Create order (blocks self-ordering)

```
DELIMITER $$

CREATE PROCEDURE pr_create_order(
  IN  p_customer_id          BIGINT,
  IN  p_service_id           BIGINT,
  IN  p_dummy_payment_status VARCHAR(20),
  OUT p_order_id             BIGINT
)
BEGIN
  DECLARE v_freelancer_id BIGINT;

  SELECT freelancer_id
    INTO v_freelancer_id
```

```
    FROM marketplace_services_service
   WHERE id = p_service_id;

  IF v_freelancer_id = p_customer_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'You cannot order your own service.';
  END IF;

  INSERT INTO orders_order (customer_id, service_id, order_date, status,
dummy_payment_status)
  VALUES (p_customer_id, p_service_id, NOW(6), 'pending', IFNULL(p_dummy_payment_status,
'paid'));

  SET p_order_id = LAST_INSERT_ID();
END$$

DELIMITER ;
```

5.6.2 pr_update_order_status — Update order status with actor-based rules

```
DELIMITER $$

CREATE PROCEDURE pr_update_order_status(
  IN p_order_id   BIGINT,
  IN p_actor_id   BIGINT,
  IN p_new_status VARCHAR(20)
)
BEGIN
  DECLARE v_customer_id   BIGINT;
  DECLARE v_freelancer_id BIGINT;

  SELECT o.customer_id, s.freelancer_id
    INTO v_customer_id, v_freelancer_id
    FROM orders_order o
    JOIN marketplace_services_service s ON s.id = o.service_id
   WHERE o.id = p_order_id;

  IF p_new_status NOT IN ('pending', 'in_progress', 'completed') THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid status.';
  END IF;

  IF p_new_status = 'in_progress' AND p_actor_id <> v_freelancer_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only freelancer can move order to in_progress.';
  END IF;
```

```
  IF p_new_status = 'completed' AND p_actor_id <> v_customer_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only customer can complete order.';
  END IF;

  UPDATE orders_order
     SET status = p_new_status
   WHERE id = p_order_id;
END$$

DELIMITER ;
```

5.6.3 pr_recompute_freelancer_rating — Recompute freelancer rating and review count

```
DELIMITER $$

CREATE PROCEDURE pr_recompute_freelancer_rating(
  IN p_freelancer_id BIGINT
)
BEGIN
  DECLARE v_avg DECIMAL(3,2);
  DECLARE v_cnt INT UNSIGNED;

  SELECT IFNULL(ROUND(AVG(r.rating), 2), 0.00), COUNT(*)
    INTO v_avg, v_cnt
    FROM orders_review r
    JOIN orders_order  o ON o.id = r.order_id
    JOIN marketplace_services_service s ON s.id = o.service_id
   WHERE s.freelancer_id = p_freelancer_id;

  UPDATE accounts_user
     SET rating       = v_avg,
         rating_count = v_cnt
   WHERE id = p_freelancer_id;
END$$

DELIMITER ;
```

5.7 Triggers

5.7.1 Canonicalize conversation user pair (BEFORE INSERT / UPDATE)

Ensures the smaller user ID is always stored in user_one_id to prevent duplicate conversations.

```
DELIMITER $$

CREATE TRIGGER tr_conv_canonical_pair_bi
BEFORE INSERT ON chat_conversation
FOR EACH ROW
BEGIN
  DECLARE v_tmp BIGINT;

  IF NEW.user_one_id = NEW.user_two_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Conversation users must be different.';
  END IF;

  IF NEW.user_one_id > NEW.user_two_id THEN
    SET v_tmp = NEW.user_one_id;
    SET NEW.user_one_id = NEW.user_two_id;
    SET NEW.user_two_id = v_tmp;
  END IF;
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER tr_conv_canonical_pair_bu
BEFORE UPDATE ON chat_conversation
FOR EACH ROW
BEGIN
  DECLARE v_tmp BIGINT;

  IF NEW.user_one_id = NEW.user_two_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Conversation users must be different.';
  END IF;

  IF NEW.user_one_id > NEW.user_two_id THEN
    SET v_tmp = NEW.user_one_id;
    SET NEW.user_one_id = NEW.user_two_id;
```

```
    SET NEW.user_two_id = v_tmp;
  END IF;
END$$

DELIMITER ;
```

5.7.2 Enforce freelancer role on service owner (BEFORE INSERT / UPDATE)

Ensures only users with role = 'freelancer' can own a service.

```
DELIMITER $$

CREATE TRIGGER tr_service_freelancer_role_bi
BEFORE INSERT ON marketplace_services_service
FOR EACH ROW
BEGIN
  DECLARE v_role VARCHAR(20);

  SELECT role INTO v_role
    FROM accounts_user
   WHERE id = NEW.freelancer_id;

  IF v_role <> 'freelancer' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Service owner must be freelancer.';
  END IF;
END$$
DELIMITER ;
DELIMITER $$

CREATE TRIGGER tr_service_freelancer_role_bu
BEFORE UPDATE ON marketplace_services_service
FOR EACH ROW
BEGIN
  DECLARE v_role VARCHAR(20);

  SELECT role INTO v_role
    FROM accounts_user
   WHERE id = NEW.freelancer_id;

  IF v_role <> 'freelancer' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Service owner must be freelancer.';
  END IF;
END$$

DELIMITER ;
```

5.7.3 Enforce customer role on order creator (BEFORE INSERT / UPDATE)

```
DELIMITER $$

CREATE TRIGGER tr_order_customer_role_bi
BEFORE INSERT ON orders_order
FOR EACH ROW
BEGIN
  DECLARE v_role VARCHAR(20);

  SELECT role INTO v_role
    FROM accounts_user
   WHERE id = NEW.customer_id;

  IF v_role <> 'customer' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Order creator must be customer.';
  END IF;
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER tr_order_customer_role_bu
BEFORE UPDATE ON orders_order
FOR EACH ROW
BEGIN
  DECLARE v_role VARCHAR(20);

  SELECT role INTO v_role
    FROM accounts_user
   WHERE id = NEW.customer_id;

  IF v_role <> 'customer' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Order creator must be customer.';
  END IF;
END$$

DELIMITER ;
```

5.7.4 Validate review: order must be completed and rating 1-5 (BEFORE INSERT / UPDATE)

```
DELIMITER $$

CREATE TRIGGER tr_review_validation_bi
BEFORE INSERT ON orders_review
FOR EACH ROW
BEGIN
  DECLARE v_status VARCHAR(20);

  SELECT status INTO v_status
    FROM orders_order
   WHERE id = NEW.order_id;

  IF v_status <> 'completed' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Review allowed only for completed orders.';
  END IF;

  IF NEW.rating < 1 OR NEW.rating > 5 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Rating must be between 1 and 5.';
  END IF;
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER tr_review_validation_bu
BEFORE UPDATE ON orders_review
FOR EACH ROW
BEGIN
  DECLARE v_status VARCHAR(20);

  SELECT status INTO v_status
    FROM orders_order
   WHERE id = NEW.order_id;

  IF v_status <> 'completed' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Review allowed only for completed orders.';
  END IF;

  IF NEW.rating < 1 OR NEW.rating > 5 THEN
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Rating must be between 1 and 5.';
  END IF;
END$$

DELIMITER ;
```

5.7.5 Recompute freelancer rating after review INSERT / UPDATE / DELETE

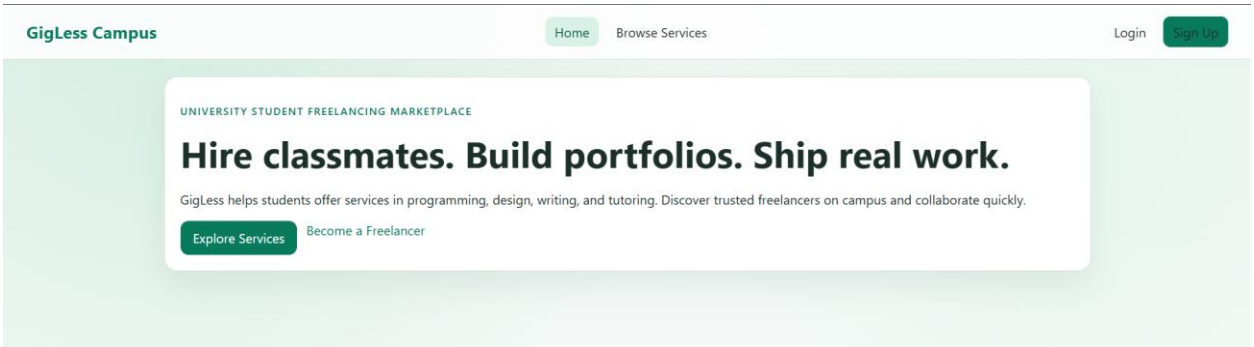
```
DELIMITER $$

CREATE TRIGGER tr_review_recalc_ai
AFTER INSERT ON orders_review
FOR EACH ROW
BEGIN
  DECLARE v_freelancer_id BIGINT;

  SELECT s.freelancer_id
    INTO v_freelancer_id
    FROM orders_order o
    JOIN marketplace_services_service s ON s.id = o.service_id
   WHERE o.id = NEW.order_id;

  CALL pr_recompute_freelancer_rating(v_freelancer_id);
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER tr_review_recalc_au
AFTER UPDATE ON orders_review
FOR EACH ROW
BEGIN
  DECLARE v_freelancer_id BIGINT;

  SELECT s.freelancer_id
    INTO v_freelancer_id
    FROM orders_order o
    JOIN marketplace_services_service s ON s.id = o.service_id
   WHERE o.id = NEW.order_id;

  CALL pr_recompute_freelancer_rating(v_freelancer_id);
END$$

DELIMITER ;
```

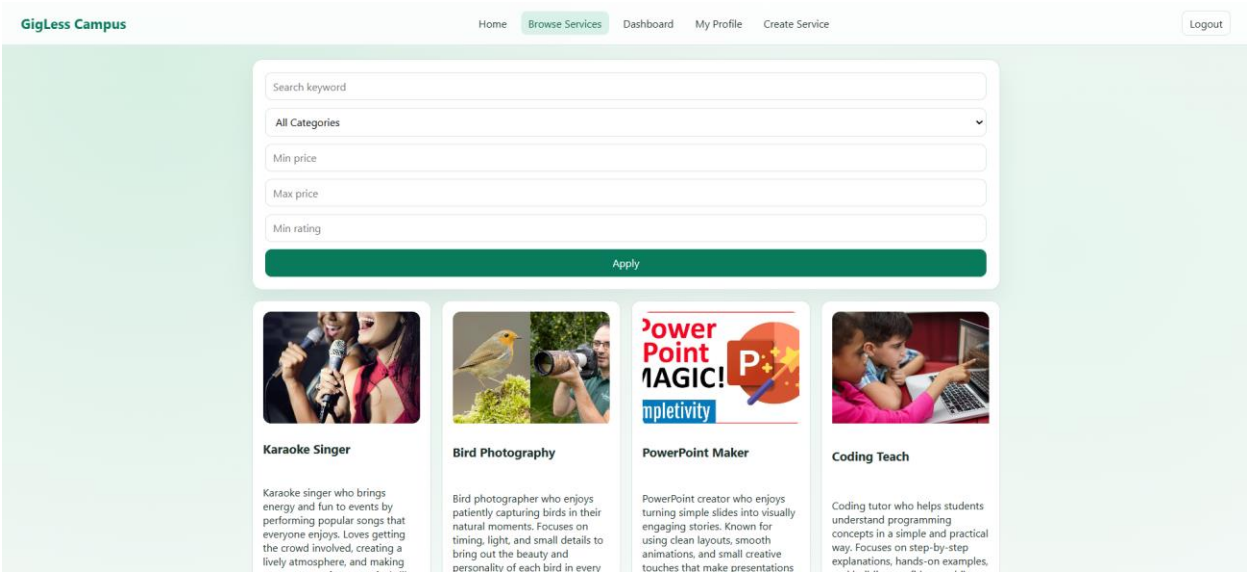
```
DELIMITER $$

CREATE TRIGGER tr_review_recalc_ad
AFTER DELETE ON orders_review
FOR EACH ROW
BEGIN
  DECLARE v_freelancer_id BIGINT;

  SELECT s.freelancer_id
    INTO v_freelancer_id
    FROM orders_order o
    JOIN marketplace_services_service s ON s.id = o.service_id
   WHERE o.id = OLD.order_id;

  CALL pr_recompute_freelancer_rating(v_freelancer_id);
END$$

DELIMITER ;
```

5.7.6 Touch conversation updated_at on new message

```
DELIMITER $$

CREATE TRIGGER tr_message_touch_conversation_ai
AFTER INSERT ON chat_message
FOR EACH ROW
BEGIN
  UPDATE chat_conversation
     SET updated_at = NOW(6)
   WHERE id = NEW.conversation_id;
END$$
```

CHAP TER 6: RESULTS & SNAPSHOTS





Search keyword

All Categories

1000

1999

Min rating

Apply

Bird Photography

Bird photographer who enjoys patiently capturing birds in their natural moments. Focuses on timing, light, and small details to bring out the beauty and personality of each bird in every shot.

Vaishnavi Bhandary $1000.00

5.00★

View details

PowerPoint Maker

PowerPoint creator who enjoys turning simple slides into visually engaging stories. Known for using clean layouts, smooth animations, and small creative touches that make presentations feel polished and memorable.

Vaishnavi Bhandary $1500.00

5.00★

View details

Website Designer

Creates clean, modern websites with a small creative twist — subtle animations, playful color accents, and tiny hidden details that make each site feel a little more alive and memorable.

VedhasGod $2000.00 0.00★

View details

VedhasGod

vedhas@gmail.com

Role: freelancer

Rating: 0.00 (0 reviews)

Bio

Website designer who creates clean, user-friendly websites with modern layouts and responsive design. Focused on simple design, good usability, and smooth user experience.

Skills (comma separated)

HTML, CSS, JavaScript, Responsive Design, UI/UX Design, Wireframing, Color Theory, Typography, Basic SEO, Website Optimization, Figma/Adobe XD, Prc

Profile Photo

Choose File No file chosen

Save Profile

Orders Received

Freelancer work requests from customers.

Website Designer
Price: $2000.00
Status: in_progress
Order Date: 4/4/2026, 4:48:11 PM

Customer: @Explioiter

Open Chat

Orders Placed

Services you purchased from other freelancers.

PowerPoint Maker
Price: $1500.00
Status: completed
Order Date: 4/4/2026, 4:53:29 PM

Customer: @vedhas

Leave 5★ Review | Open Chat

Conversations

Shreyas Nayak
@ChinnappaSwami
Bird Photography

VedhasGod
@vedhas
PowerPoint Maker

Chat about PowerPoint Maker

Live connected

VedhasGod 4/4/2026, 4:54:37 PM
i wanted a powerpoint for a hackathon i am in

VedhasGod 4/4/2026, 4:54:51 PM
its about database

VedhasGod 4/4/2026, 4:54:59 PM
the topic is Gigless

VedhasGod 4/4/2026, 4:55:06 PM
and it works on making a freelancing website

Vaishnavi Bhandary 4/4/2026, 4:57:16 PM
ah

Vaishnavi Bhandary 4/4/2026, 4:57:17 PM
okok

Vaishnavi Bhandary 4/4/2026, 4:57:26 PM
illl work on it in a few fdays

Vaishnavi Bhandary 4/4/2026, 4:57:36 PM
i seem to be busy in a bird watching trip

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

GigLess successfully demonstrates a fully functional campus freelancing marketplace built on modern web technologies. The system achieves all its primary objectives: role-based user management, service listing and discovery, a structured order lifecycle, an automated review and rating system, and real-time communication between participants. The relational database schema is well-normalized, with referential integrity enforced through foreign key constraints and business logic automated via model-level triggers.

The project validated that Django REST Framework, combined with Django Channels for WebSocket support and React for the frontend, provides a robust and developer-friendly full-stack architecture for marketplace applications. The use of JWT authentication with HTTP-only cookies ensures a secure and stateless session management approach suitable for production deployment.

7.2 Limitations

Despite its functionality, the current implementation has several limitations:

1. Mock Payment System — The payment flow is simulated via a dummy_payment_status field. No real payment gateway (e.g., Razorpay, Stripe) is integrated, making the platform unsuitable for actual financial transactions in its current state.
2. In-Memory Channel Layer — Django Channels currently uses an in-memory channel layer, which is suitable only for single-server local development. It does not support horizontal scaling or persistence across server restarts.

3. No Admin Dashboard — There is no dedicated admin panel for moderating services, resolving disputes, or managing flagged content beyond Django's default admin interface.
4. No Notification System — Users are not notified of order status changes, new messages, or new reviews outside of the chat interface.
5. No Email Verification — User registration does not require email confirmation, leaving the platform open to fake accounts.

7.3 Future Work

The following enhancements are proposed for future iterations of GigLess:

1. Payment Gateway Integration — Integrate a real payment provider (e.g., Razorpay for Indian campuses) to enable actual transactions, escrow holding, and refund workflows.
2. Redis Channel Layer — Replace the in-memory channel layer with Redis to enable scalable, persistent WebSocket connections suitable for production deployment.
3. Push Notifications — Implement a notification system (via Firebase Cloud Messaging or server-sent events) to alert users of order updates and new messages.
4. Mobile Application — Develop a React Native companion app to improve accessibility and engagement on mobile devices, which are the primary devices for most students.
5. Dispute Resolution — Build a formal dispute resolution workflow where unresolved order conflicts can be escalated to platform administrators.
6. Email Verification & Password Reset — Add email-based account verification on signup and a password reset flow to improve account security and authenticity.
7. Advanced Search & Recommendations — Implement full-text search and a recommendation engine that suggests services to customers based on browsing history and peer activity.

CHAPTER 8: REFERENCES

[1] Fiverr International Ltd. (2024). *Fiverr — Freelance services marketplace*. https://www.fiverr.com

[2] Upwork Global Inc. (2024). *Upwork — How it works*. https://www.upwork.com/work/

[3] Django Software Foundation. (2024). *Django documentation (Version 5.0)*. https://docs.djangoproject.com/en/5.0/

[4] Encode OSS. (2024). *Django REST Framework documentation*. https://www.django-rest-framework.org/

[5] Django Channels contributors. (2024). *Django Channels documentation*. https://channels.readthedocs.io/en/stable/

[6] Meta Open Source. (2024). *React documentation*. https://react.dev/

[7] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (RFC 7519)*. IETF. https://datatracker.ietf.org/doc/html/rfc7519

[8] Oracle Corporation. (2024). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/